

MRP-r and MRP-p

Qianyu Dong

2025-12-09

Contents

Data Overview	1
Model Specification	2
Multilevel regression	2
Poststratification	4
Fit Model	7
Results	9
Convergence diagnostic (rhat)	9
PUMA level estimates	9
Metrics	9

Data Overview

1. Population data `acs_pop`.
2. One set of PS `ps_df` and NPS `nps_df` by `source("code/tests/create_test_sample.R")`

Samples regenerated

```
data.frame(  
  Sample = c("Population", "PS", "NPS"),  
  N = c(nrow(acs_pop), nrow(ps_df), nrow(nps_df)),  
  HICOV_Rate = c(mean(acs_pop$PUBCOV), mean(ps_df$PUBCOV), mean(nps_df$PUBCOV))  
)
```

```
##      Sample      N HICOV_Rate  
## 1 Population 1475631 0.6030925  
## 2         PS      7307 0.6194060  
## 3         NPS      72848 0.4360861
```

Model Specification

Multilevel regression

Let i be individual, j be PUMA. We first fit logistic regression using NPS data.

$$\begin{aligned}y_i &\sim \text{Bernoulli}(p_i) \\ \text{logit}(p_i) &= \mathbf{x}_i^\top \beta \\ \beta &\sim \text{Normal}(0, \sigma_\beta)\end{aligned}$$

covariates:

- **Age** (categorized as 1. under 18, 2. 18-33, 3. 34-64, and 4. 65+)
- **race** (Hispanic, white, Asian, Black, other), and
- **sex** (male, female)

priors:

Moderately informative priors $\sigma_\beta = 2.5 * sd(Y)/sd(X)$ are used for covariates based on prior specification in supplementary A.1.

```
getMRP=function(MR=nps,
                 ps=ps,
                 acs_pop=acs_pop,
                 WFPBB=FALSE,
                 L=5){

  # TRAINING DATA ----
  nps_ca <- MR # nps

  # lock factor levels from training
  nps_ca$AGEP_binned <- factor(nps_ca$AGEP_binned)
  nps_ca$SEX <- factor(nps_ca$SEX)
  nps_ca$RAC1P <- factor(nps_ca$RAC1P)
  PUMA_lev <- levels(factor(nps_ca$PUMA)) # lock PUMA levels

  # fixed-effects design
  X_train <- model.matrix(~ 0 + AGEP_binned + SEX + RAC1P, data = nps_ca)
  k <- ncol(X_train); n <- nrow(X_train)
  y <- as.integer(nps_ca$PUBCOV)

  # n <- nrow(X_train)
  # threads <- 4
  # grainsize=as.integer(n / (threads))

  # data dependent prior for beta
  sd_y <- stats::sd(as.numeric(y))
  sd_x <- apply(X_train, 2, stats::sd)
  sd_x[sd_x == 0] <- NA_real_ # or small number
  beta_scale <- 2.5 * sd_y / sd_x
```

```

stan_data <- list(
  # training
  n = n,
  k = k,
  X = X_train,
  y = y,
  grainsize = 16000,
  beta_scale = as.vector(beta_scale)
)

```

```

fit <- mod$sample(
  data = stan_data,
  chains = 2,
  parallel_chains = 2,
  threads_per_chain = 4,
  iter_warmup = 500,
  iter_sampling = 1000,
  seed = 123
)

```

```

sum_tbl <- fit$summary()

```

```

.....

```

```

functions {
  real bern_glm_ll_slice(array[] int y_slice, int start, int end,
                        matrix X,
                        vector beta) {
    // intercept = 0 because X is built with ~ 0 + AGE + SEX + RAC1P
    return bernoulli_logit_glm_lpmf(y_slice | X[start:end, ], 0, beta);
  }
}

```

```

data {
  int<lower=1> n;
  int<lower=1> k;
  matrix[n, k] X;
  array[n] int<lower=0, upper=1> y;
  int<lower=1> grainsize;
  vector<lower=0>[k] beta_scale;
}

```

```

parameters {
  vector[k] beta;
}

```

```

model {
  // Priors: _j ~ Normal(0, beta_scale[j])
  beta ~ normal(0, beta_scale);

  // Likelihood

```

```
target += reduce_sum(bern_glm_ll_slice, y, grainsize, X, beta);
}
```

Poststrification

using probability sample

Let s be posterior draw and c be cell build from probability sample, w_c is sum of individual weight in the cell c .

$$p_{c,s} = \text{expit}(X_c^\top \beta_s)$$

The PUMA-level estimate for area j is then

$$\theta_{j,s} = \frac{\sum_{c \in j} w_c p_{c,s}}{\sum_{c \in j} w_c}.$$

using population

The same code are used, but cell is build using whole population i.e. `acs_pop`.

```
# ----- 1) Individuals to cells -----
.align_to_train <- function(df, train_ref, PUMA_lev) {
  df$AGEP_binned <- factor(df$AGEP_binned, levels = levels(train_ref$AGEP_binned))
  df$SEX <- factor(df$SEX, levels = levels(train_ref$SEX))
  df$RAC1P <- factor(df$RAC1P, levels = levels(train_ref$RAC1P))
  df$PUMA <- factor(df$PUMA, levels = PUMA_lev)
  df
}

collapse_to_cells <- function(df, train_ref, PUMA_lev,
                             design_formula = ~ 0 + AGEP_binned + SEX + RAC1P,
                             weight_col = NULL) {
  df <- .align_to_train(df, train_ref, PUMA_lev)
  w <- if (!is.null(weight_col) && weight_col %in% names(df)) df[[weight_col]] else 1
  w <- as.numeric(w); w[!is.finite(w) | w < 0] <- 0

  cells <- df |>
    dplyr::mutate(W = w) |>
    dplyr::group_by(AGEP_binned, SEX, RAC1P, PUMA) |>
    dplyr::summarise(w = sum(W), .groups = "drop")

  Xp <- model.matrix(design_formula, data = cells)
  g <- as.integer(cells$PUMA) # 1..J
  list(Xp = Xp, g = g, w = cells$w, cells = cells)
}

# ----- 2) Extract CmdStanR draws -----
get_beta_draws <- function(fit) {
  # S x K (format='matrix' gives draws as rows)
  fit$draws("beta", format = "matrix")
}
```

```

}

# ----- 3) poststrat by cell -----
poststrat_SxJ <- function(beta, Xp, g, w = NULL, J = max(g)) {
  # beta: S x K (posterior draws)
  # Xp : N x K (prediction design matrix)
  # g : length-N integer group indices 1..J
  # w : length-N weights (if NULL, all 1)
  # J : number of groups (PUMAs)

  N <- nrow(Xp)
  S <- nrow(beta)

  # weights
  if (is.null(w)) w <- rep(1, N) else w <- as.numeric(w)
  w[!is.finite(w) | w < 0] <- 0

  # 1) Compute probabilities for *all cells x all draws*
  # result: (N x S)
  eta <- Xp %*% t(beta)
  p <- plogis(eta)

  # 2) Pre-split indices and normalize weights by PUMA
  idx_by_j <- split(seq_len(N), factor(g, levels = seq_len(J)))
  w_norm <- lapply(idx_by_j, function(idx) {
    if (length(idx) == 0) return(numeric(0))
    sw <- sum(w[idx])
    if (sw > 0) w[idx] / sw else rep(0, length(idx))
  })

  # 3) Aggregate weighted means for each PUMA (S x J)
  mu <- matrix(NA_real_, nrow = S, ncol = J)
  for (j in seq_len(J)) {
    idx <- idx_by_j[[j]]
    if (length(idx) == 0) next
    wj <- w_norm[[j]]
    mu[, j] <- as.numeric(crossprod(wj, p[idx, , drop = FALSE]))
  }

  mu # (S x J)
}

getMRP=function(MR=nps,
                ps=ps,
                acs_pop=acs_pop,
                WFPBB=FALSE,
                L=5
                ){
.....

```

```

# TRAINING ref (to lock levels & PUMA order)
nps_ca <- MR
nps_ca$AGEP_binned <- factor(nps_ca$AGEP_binned)
nps_ca$SEX <- factor(nps_ca$SEX)
nps_ca$RAC1P <- factor(nps_ca$RAC1P)
PUMA_lev <- levels(factor(nps_ca$PUMA))
J <- length(PUMA_lev)

# Collapse Ps and Pop to cells (huge speed/memory win)
ps_cells <- collapse_to_cells(ps, nps_ca, PUMA_lev, weight_col = "PWGTP")
pop_cells <- collapse_to_cells(acs_pop, nps_ca, PUMA_lev, weight_col = "PWGTP")

beta <- get_beta_draws(fit) # S x K

mu_ps_draws <- if (nrow(ps_cells$Xp) > 0)
  poststrat_SxJ(beta, ps_cells$Xp, ps_cells$g, ps_cells$w, J) else
  matrix(NA_real_, nrow = nrow(beta), ncol = J)

mu_pop_draws <- if (nrow(pop_cells$Xp) > 0)
  poststrat_SxJ(beta=beta,
    Xp=pop_cells$Xp,
    g=pop_cells$g,
    w=pop_cells$w,
    J=J) else
  matrix(NA_real_, nrow = nrow(beta), ncol = J)

puma_summary_mrpr <- make_summary(mu_ps_draws, PUMA_lev, "mrp-r")
puma_summary_mrpp <- make_summary(mu_pop_draws, PUMA_lev, "mrp-p")

if(WFPBB){
  index_list <- vector("list", L)
  system.time({
    for (l in 1:L) {
      index_list[[l]] <- MSIMST::WFPBB(
        y = 1:nrow(ps),
        w = ps$PWGTP,
        N = sum(ps$PWGTP),
        n = nrow(ps),
        verbatim = FALSE
      )
      print(paste("Now generate WFPBB", l))
    }
  })
  ps_star <- ps[unlist(index_list), ]
  ps_star_cells <- collapse_to_cells(ps_star, nps_ca, PUMA_lev, weight_col = "PWGTP")

  mu_star_draws <- if (nrow(ps_star_cells$Xp) > 0)
    poststrat_SxJ(beta=beta,
      Xp=ps_star_cells$Xp,
      g=ps_star_cells$g,

```

```

        w=ps_star_cells$w,
        J=J) else
        matrix(NA_real_, nrow = nrow(beta), ncol = J)

puma_summary_mrpr_WFPBB <- make_summary(mu_star_draws, PUMA_lev, "mrp-r-WFPBB")

}else{
  puma_summary_mrpr_WFPBB=NULL
}

return(list(
  puma_summary_mrpr=puma_summary_mrpr,
  puma_summary_mrpp=puma_summary_mrpp,
  puma_summary_mrpr_WFPBB=puma_summary_mrpr_WFPBB,
  rhat=sum_tbl$rhat
))
}

```

Fit Model

```

# Profile getMRP function
#use ps$weights for PWGTP
ps_df$PWGTP=ps$weights

cat("   Profiling getMRP()...\n")

```

```
##   Profiling getMRP()...
```

```

start_time <- Sys.time()

tryCatch(
{
  mrp <- getMRP(
    MR = nps_df,
    ps = ps_df,
    acs_pop = acs_pop,
    WFPBB=F,
    L=5
  )

  mrp_time <- Sys.time() - start_time
  cat("  getMRP completed in", round(mrp_time, 2), attr(mrp_time, "units"), "\n")

  # Post Stratification results (MRP-R & MRP-P)
  mrpra <- mrp$puma_summary_mrpr
  mrppb <- mrp$puma_summary_mrpp
  mrppc <- mrp$puma_summary_mrpr_WFPBB

```

```

    cat("- MRP-R results:", nrow(mrpra), "PUMAs\n")
    cat("- MRP-P results:", nrow(mrppb), "PUMAs\n")
  },
  error = function(e) {
    cat(" getMRP failed:", e$message, "\n")
  }
)

```

```
## Running MCMC with 2 parallel chains, with 4 thread(s) per chain...
```

```
##
## Chain 1 Iteration:    1 / 1500 [  0%] (Warmup)
## Chain 2 Iteration:    1 / 1500 [  0%] (Warmup)
## Chain 2 Iteration:  100 / 1500 [  6%] (Warmup)
## Chain 1 Iteration:  100 / 1500 [  6%] (Warmup)
## Chain 2 Iteration:  200 / 1500 [ 13%] (Warmup)
## Chain 1 Iteration:  200 / 1500 [ 13%] (Warmup)
## Chain 2 Iteration:  300 / 1500 [ 20%] (Warmup)
## Chain 1 Iteration:  300 / 1500 [ 20%] (Warmup)
## Chain 2 Iteration:  400 / 1500 [ 26%] (Warmup)
## Chain 1 Iteration:  400 / 1500 [ 26%] (Warmup)
## Chain 2 Iteration:  500 / 1500 [ 33%] (Warmup)
## Chain 2 Iteration:  501 / 1500 [ 33%] (Sampling)
## Chain 1 Iteration:  500 / 1500 [ 33%] (Warmup)
## Chain 1 Iteration:  501 / 1500 [ 33%] (Sampling)
## Chain 2 Iteration:  600 / 1500 [ 40%] (Sampling)
## Chain 1 Iteration:  600 / 1500 [ 40%] (Sampling)
## Chain 2 Iteration:  700 / 1500 [ 46%] (Sampling)
## Chain 1 Iteration:  700 / 1500 [ 46%] (Sampling)
## Chain 2 Iteration:  800 / 1500 [ 53%] (Sampling)
## Chain 1 Iteration:  800 / 1500 [ 53%] (Sampling)
## Chain 2 Iteration:  900 / 1500 [ 60%] (Sampling)
## Chain 1 Iteration:  900 / 1500 [ 60%] (Sampling)
## Chain 2 Iteration: 1000 / 1500 [ 66%] (Sampling)
## Chain 1 Iteration: 1000 / 1500 [ 66%] (Sampling)
## Chain 2 Iteration: 1100 / 1500 [ 73%] (Sampling)
## Chain 1 Iteration: 1100 / 1500 [ 73%] (Sampling)
## Chain 2 Iteration: 1200 / 1500 [ 80%] (Sampling)
## Chain 1 Iteration: 1300 / 1500 [ 86%] (Sampling)
## Chain 2 Iteration: 1200 / 1500 [ 80%] (Sampling)
## Chain 1 Iteration: 1400 / 1500 [ 93%] (Sampling)
## Chain 2 Iteration: 1300 / 1500 [ 86%] (Sampling)
## Chain 1 Iteration: 1500 / 1500 [100%] (Sampling)
## Chain 1 finished in 6.5 seconds.
## Chain 2 Iteration: 1400 / 1500 [ 93%] (Sampling)
## Chain 2 Iteration: 1500 / 1500 [100%] (Sampling)
## Chain 2 finished in 6.8 seconds.
##
## Both chains finished successfully.
## Mean chain execution time: 6.6 seconds.
## Total execution time: 7.0 seconds.
##
## getMRP completed in 8.32 secs

```



```
## - MRP-R results: 281 PUMAs
## - MRP-P results: 281 PUMAs
```

Results

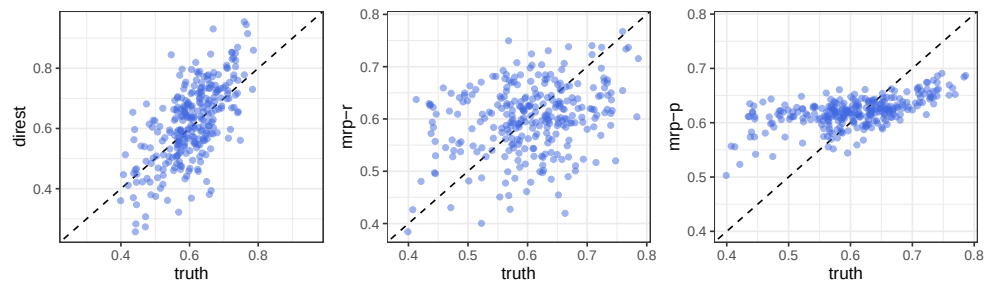
Convergence diagnostic (rhat)

Close to 1 suggests good convergence.

```
max(mrp$rhat)
```

```
## [1] 1.003361
```

PUMA level estimates



Metrics

```
## # A tibble: 3 x 5
##   model      MSE    MAB Coverage `Int. Score`
##   <chr>    <dbl> <dbl>    <dbl>      <dbl>
## 1 direct 0.00946 0.0779  0.929      0.494
## 2 mrp-p  0.00477 0.0542  0.0498     2.00
## 3 mrp-r  0.00775 0.0710  0.0498     2.66
```